

Cursed battle

Rapport de projet

Romain STEFANI

Guillaume FAURE

23 mai 2026

Games on web 2026 - <https://www.cgi.com/france/fr-fr/event/games-on-web-2026>

Table des matières

1	Introduction	2
1.1	L'idée du jeu	2
1.2	La jouabilité recherchée	2
1.3	Le style	2
2	Architecture	2
2.1	Organisation du code	2
2.2	Flux d'exécution et communication entre composants	2
2.3	Techniques et patrons de développement	3
2.4	Bibliothèques utilisées	3
3	Game design	3
3.1	Les maps 3D	3
3.2	Les animations	3
3.3	Les personnages 3D	4
3.4	Le son	4
3.5	Les détails graphiques	4
4	Jouabilité du joueur	4
4.1	Les contrôles : clavier, manette et paramétrage	4
4.2	Le fonctionnement de la machine à états	4
4.3	Le fonctionnement des collisions	5
4.4	Les attaques disponibles	5
5	Conclusion	5

1 Introduction

1.1 L'idée du jeu

Notre projet est un jeu de combat en 2.5D : le gameplay se déroule sur un seul axe, comme dans un jeu de combat classique, tandis que le rendu est entièrement en 3D. Le jeu se joue actuellement à deux joueurs en local. L'un des objectifs avant l'évaluation du concours est d'ajouter un adversaire contrôlé par une intelligence artificielle avec un réseau de neurones. Mais pour l'instant, seul le mode deux joueurs est implémenté.

1.2 La jouabilité recherchée

Nous nous sommes inspirés de Street Fighter pour la jouabilité : des déplacements et des sauts (avant, arrière, vertical), une grande liste de coups, un système de garde et une gestion du stun. L'objectif est d'obtenir des combats dynamique et prenant avec une jouabilité variée.

1.3 Le style

Le jeu adopte un style type anime, dans l'esprit de Demon Slayer et Jujutsu Kaisen et pour l'audio on s'est aussi inspiré de Street Fighter.

2 Architecture

Avant de présenter notre architecture, il est important de préciser notre usage de l'intelligence artificielle durant le développement. Nous l'avons utilisée comme architecte, afin de partir sur une base propre et bien structurée, ainsi que pour répondre à certaines questions. Nous avons en revanche choisi de développer nous-mêmes la quasi-totalité du jeu, afin de garder le contrôle sur l'avancement du projet et de pouvoir nous y retrouver facilement.

2.1 Organisation du code

Le code source est organisé par fonctionnalités dans le dossier `src/`. Chaque domaine est isolé dans son propre dossier :

- `core/` : infrastructure partagée (`Game`, `SceneManager`, `AssetManager`, `InputManager`, `EventBus`, `SettingsManager`, etc.)
- `scenes/` : les écrans du jeu (chargement, menu, sélection des personnages, combat)
- `character/` : les personnages et leur machine à états
- `combat/` : les règles du combat
- `arena/` : les décors de combat et la caméra
- `audio/`, `weather/`, `ui/`, `utils/` et `ai/` pour les autres systèmes.

Ce découpage a été choisi volontairement pour faciliter l'ajout de nouvelles fonctionnalités : par exemple, ajouter un nouvel état de personnage revient à créer un fichier dans `character/state/states/`, et ajouter un coup se fait de manière isolée sans toucher au reste du jeu.

2.2 Flux d'exécution et communication entre composants

Le démarrage et le fonctionnement du jeu suivent une chaîne claire :

- `main.js` instancie la classe `Game`
- `Game` initialise le moteur Babylon.js, l'`AssetManager`, charge le moteur physique Havok, puis crée le `SceneManager`
- le `SceneManager` orchestre les écrans du jeu (chargement, menu, sélection des personnages, combat) et gère les transitions entre eux

- lorsqu'un combat démarre, la **FightScene** construit à son tour les éléments du combat : l'**Arena**, les deux **Player**, le **CombatSystem**, le **MatchManager** et l'**UIManager**
- à chaque image, la boucle de rendu de **Game** appelle le **SceneManager**, qui délègue la mise à jour à la scène active celle-ci met à jour les joueurs (et donc leur machine à états), le système de combat, le timer du match et l'interface.

2.3 Techniques et patrons de développement

Le développement s'appuie sur plusieurs patrons de conception, choisis pour leur capacité à faire évoluer le projet sans tout réorganiser :

- **Machine à états** : le comportement de chaque personnage est géré par une machine à états (détaillée en section 4.2).
- **Patron observateur** : la communication entre composants passe par des observables et un bus d'événements (**EventBus**). Les composants n'ont pas besoin de se connaître directement : un émetteur signale un événement et les composants intéressés y réagissent. Cela réduit fortement le couplage et permet d'ajouter ou de retirer un système sans modifier les autres. Utilisé notamment pour les collisions lors des attaques.
- **Patron manager** : les responsabilités transverses sont centralisées dans des gestionnaires dédiés (**SceneManager** pour les écrans, **AssetManager** pour les ressources, etc.). Chaque manager a une responsabilité claire, ce qui évite de disperser cette logique dans tout le code.
- **Séparation des fonctionnalités** : chaque domaine est isolé dans son propre dossier et communique avec les autres via des interfaces simples, ce qui rend le projet plus lisible et plus facile à faire évoluer.
- **Gestion des collisions** : les collisions de déplacement et d'attaque reposent sur des techniques décrites en section 4.3.

2.4 Bibliothèques utilisées

- **Babylon.js** : moteur de rendu 3D.
- **Havok** : moteur physique (gravité, collisions, knockback).
- **Vite** : outil de build et serveur de développement.
- **TensorFlow.js** : prévu pour l'intelligence artificielle à venir.

3 Game design

3.1 Les maps 3D

Pour les arènes, nous sommes partis de modèles 3D de décors existants, que nous avons ensuite modifiés, assemblés et nettoyés dans Blender afin d'obtenir des maps cohérentes avec l'univers de notre jeu. La principale difficulté a été la taille des maps : un décor trop volumineux ou comportant trop de détails fait rapidement chuter le nombre d'images par seconde. Une part importante du travail a donc consisté à alléger les maps pour préserver de bonnes performances.

3.2 Les animations

Pour les animations, nous avons utilisé celles de Mixamo, ce qui nous a permis de disposer d'une large base d'animations reposant toutes sur le même squelette. Nous avons ajouté ces animations à chaque personnage dans Blender, sans difficulté majeure : nous l'avions déjà fait pour le jeu de l'année précédente. La partie la plus délicate n'a pas été d'ajouter les animations au squelette, mais de faire en sorte que le squelette soit correctement relié au mesh du personnage et que les animations rendent bien visuellement.

3.3 Les personnages 3D

Le travail sur les personnages 3D a représenté une part importante du projet et a entièrement été réalisé dans Blender.

La première étape a consisté à trouver de bons modèles 3D. Nous en avons récupéré sur internet qui nous plaisaient beaucoup et qui étaient très bien réalisés, mais qui n'étaient malheureusement pas conçus pour le jeu vidéo : ils comportaient beaucoup trop de détails et de complexité, au point qu'une fois importés dans le jeu, ce n'était plus jouable. Notre premier travail a donc été de nettoyer au maximum les mesh, en fusionnant le plus de détails possible pour les simplifier.

Une fois une version moins volumineuse obtenue, il fallait y ajouter les animations. Le problème était que ces modèles ne provenaient pas de Mixamo et n'utilisaient donc pas le même squelette. Notre première idée a été de conserver le squelette d'origine du modèle, qui fonctionnait parfaitement avec le mesh, et de relier à celui-ci les os du squelette Mixamo, nous nous sommes cependant heurtés à de nombreux problèmes. Nous avons alors changé de stratégie : retirer le squelette d'origine du modèle et le remplacer par celui de Mixamo. Sans entrer dans le détail, cette opération a été très longue et très complexe, mais elle nous a finalement permis d'obtenir un squelette Mixamo dans un modèle issu d'internet, tout en conservant suffisamment de détails pour que le rendu reste beau visuellement et que le jeu reste performant.

3.4 Le son

Pour le son, nous avons dû reprendre nous-mêmes en JavaScript le fonctionnement proposé par Babylon.js, que nous ne parvenions pas à faire fonctionner correctement. Une fois ce système en place, nous avons pu ajouter les sons souhaités et les déclencher au bon moment dans le jeu, nous nous sommes ici aussi inspirés de Street Fighter. Le système audio est organisé en plusieurs canaux distincts pour la musique, les effets sonores et les voix. Coordonnés par un mixeur audio, ce qui permet de gérer indépendamment chaque type de son.

3.5 Les détails graphiques

Pour apporter davantage de réalisme, nous avons ajouté des ombres sur les personnages ainsi que plusieurs effets visuels : de la pluie qui tombe et forme des flaques d'eau, et des animations lorsque le personnage marche dans l'eau, des petites vagues partent alors de son talon à chaque pas. Ce sont ces petits détails qui rendent le jeu plus agréable à jouer.

4 Jouabilité du joueur

4.1 Les contrôles : clavier, manette et paramétrage

Le jeu se joue au clavier ou à la manette. Chaque joueur peut en plus personnaliser ses touches via les paramètres.

4.2 Le fonctionnement de la machine à états

Le comportement de chaque personnage repose sur une machine à états. Nous avons choisi ce patron parce qu'il correspond naturellement à un jeu de combat : à tout instant, un personnage se trouve dans une situation bien précise (immobile, en marche, en saut, entrain d'attaquer, en garde, stun, etc.) et ne peut pas être dans deux situations à la fois.

Un seul état est actif à la fois, et chaque état gère sa propre logique (animation à jouer, minutage, et pour les attaques l'activation de la hitbox). La machine se contente de coordonner

l'ensemble et de gérer les transitions. À chaque image, elle met à jour l'orientation du personnage puis traite les entrées du joueur selon un ordre de priorité (les attaques d'abord, puis les déplacements et les sauts, et la position neutre par défaut).

Ce fonctionnement rend le système clair et facile à étendre : ajouter un nouveau coup ou un nouveau comportement revient simplement à créer un nouvel état. Les états implémentés couvrent l'ensemble des situations du combat : idle, avancer et reculer, sauter (vertical, avant et arrière), garde, stun, chute, attaques, ainsi que les états de victoire et de défaite.

4.3 Le fonctionnement des collisions

On distingue deux types de collisions.

Les collisions entre personnages sont gérées par le moteur physique Havok. Chaque personnage est représenté par un corps physique de forme capsule, contraint sur le plan de jeu. Havok empêche ainsi les deux personnages de se traverser et gère leurs déplacements, la gravité et les forces qui leur sont appliquées.

Les collisions des coups ont demandé plus de réflexion. Nous avons testé plusieurs méthodes mais on a fini avec une plutôt simple, nous attachons une sphère de collision à l'os correspondant, par exemple : la main gauche pour le jab, le pied droit pour le coup de pied léger, etc. Cette sphère ne sert qu'à détecter la collision et suit naturellement le mouvement de l'os pendant l'animation. Cette approche nous évite d'avoir à vérifier manuellement, à un instant précis de l'animation, si l'adversaire se trouve au bon endroit. C'est le mouvement de l'animation lui-même qui place la sphère. Elle n'est active que pendant la fenêtre utile du coup, définie par un intervalle d'images. Lorsque cette sphère entre en collision avec la capsule de l'adversaire, le système de combat applique les dégâts et le stun, et une force est appliquée à l'adversaire pour le repousser vers l'arrière.

4.4 Les attaques disponibles

Nous avons ajouté de plus en plus de coups au fur et à mesure du développement et on se retrouve avec la liste finale suivante :

- **Jab** : coup de poing rapide, peu de dégâts.
- **Cross** : coup de poing puissant.
- **Coup de pied léger** : coup de pied rapide.
- **Coup de pied lourd** : coup de pied puissant.
- **Balayage** : attaque basse qui fait chuter l'adversaire.
- **Boule de feu** : projectile tiré devant le personnage.

5 Conclusion

Nous sommes satisfaits du résultat obtenu. Le jeu est agréable à jouer et visuellement réussi, on a passé beaucoup de temps sur blender et ça se voit sur le rendu, c'est également les petits détails comme la pluie, les ombres et les effets visuels qui rendent le jeu plus immersif. La prochaine étape est d'ajouter l'ia dans l'adversaire ainsi que des maps supplémentaires.

Le jeu est hébergé ici : <https://games-on-web2026.vercel.app/>